

CFPB Complaint Classifier

Project Overview Document

Consumer Financial Protection Bureau (CFPB)
Automated Consumer Complaint Classification System

Prepared for Interview Reference

Date: June 2026

Table of Contents

- 1. Project Overview - What is this project about?
- 2. Why We Built This - The Problem We Solved
- 3. Project Structure - Files and Folders
- 4. Data Source - Where the Data Came From
- 5. Notebook 1: Exploratory Data Analysis (EDA)
- 6. Notebook 2: Data Preprocessing
- 7. Notebook 3: Model Training & Evaluation
- 8. Web Application (Streamlit App)
- 9. Results and Performance Metrics
- 10. Technology Stack Used
- 11. Key Interview Talking Points

CFPB Complaint Classifier - Project Overview

1. Project Overview

This project is an automated system that classifies consumer complaints submitted to the Consumer Financial Protection Bureau (CFPB). The CFPB is a US government agency that handles complaints from consumers about financial products and services like credit cards, debt collection, mortgages, etc.

The system takes a free-text complaint written by a consumer and automatically categorizes it into one of two categories: "Debt Collection" or "Credit Card". This is done using two different approaches side by side:

- Traditional Machine Learning (ML): TF-IDF + Logistic Regression
- Large Language Model (LLM): Google Gemini 2.5 Flash

The project includes a beautiful dark-themed web interface built with Streamlit where users can paste any complaint and get instant predictions from both models, along with performance analytics and visualizations.

2. Why We Built This - The Problem

The CFPB receives thousands of unstructured, free-text consumer complaints every single day. Humans have to manually read each complaint, understand what it is about, and then route it to the correct department or financial institution.

This manual process has several problems:

- It is very slow - thousands of complaints pile up
- It is labor-intensive - requires many human workers
- It is prone to human error - people make mistakes when tired
- It is inconsistent - different people may classify the same complaint differently

Our project solves this by building an automated system that can instantly classify any complaint into the correct category. We also compared two approaches - a fast, cheap traditional ML model versus a powerful but slower LLM - to show the trade-offs between different AI approaches.

CFPB Complaint Classifier - Project Overview

3. Project Structure - Files and Folders

Here is what each file and folder in the project does:

data/complaints.csv

The raw dataset downloaded from CFPB. Contains 114,953 consumer complaints with 16 columns including product type, complaint narrative, company name, state, etc.

data/cleaned_train.csv

Preprocessed training dataset. Contains 6,400 balanced complaints (3,200 Debt Collection + 3,200 Credit Card) used to train the ML model.

data/cleaned_test.csv

Preprocessed test dataset. Contains 1,600 balanced complaints (800 Debt Collection + 800 Credit Card) used to evaluate the ML model.

notebooks/01_eda.ipynb

Exploratory Data Analysis notebook. Loads raw data, analyzes class distribution, checks for duplicates, redacted text, word counts, and generates visualizations.

notebooks/02_preprocessing.ipynb

Data Preprocessing notebook. Cleans the raw data by removing duplicates, null values, redacted text, short narratives. Balances classes and creates train/test split.

notebooks/03_model_training.ipynb

Model Training notebook. Builds TF-IDF vectorizer and Logistic Regression model. Evaluates performance. Also tests Google Gemini LLM on a 300-row sample.

app/app.py

Streamlit web application. Provides a dark-themed UI where users paste complaints and get predictions from both ML and LLM models. Shows analytics dashboard.

a.py

A simple test script to verify the Google Gemini API connection is working.

outputs/logistic_model.joblib

The trained Logistic Regression model saved using joblib for reuse.

outputs/tfidf_vectorizer.joblib

The trained TF-IDF vectorizer saved using joblib for reuse.

outputs/metrics.json

JSON file containing the evaluation metrics (accuracy, precision, recall, F1) for both ML and LLM models.

outputs/ml_predictions.csv

CSV file containing the ML model predictions on the test set.

outputs/*.png

Various visualization images: confusion matrix, class balance charts, word count distributions, etc.

4. Data Source

The data comes from the Consumer Financial Protection Bureau (CFPB) public database. The CFPB collects complaints from consumers about financial products and services in the United States.

About the raw dataset:

- Total rows: 114,953 complaints
- Total columns: 16 (Product, Sub-product, Issue, Narrative, Company, State, etc.)
- Two product categories: Debt Collection (79,308) and Credit Card (35,645)
- The key column we use is "Consumer complaint narrative" - the free text written by consumers

We focused on only two categories (Debt Collection vs Credit Card) to keep the problem manageable while still demonstrating the full ML pipeline.

5. Notebook 1: Exploratory Data Analysis

The first notebook (01_eda.ipynb) is where we explore and understand the raw data before doing any cleaning or modeling. Here is what we found:

5.1 Dataset Size

Loaded 114,953 rows with 16 columns. The dataset has complaints from consumers across the US.

5.2 Missing Values

Found missing values in several columns:

- Tags: 88% missing (we do not need this column)
- Company public response: 63% missing (not needed)
- State: 0.25% missing (284 rows - filled with most common value)
- Sub-product, Issue, Company: very few nulls

5.3 Class Imbalance

Debt Collection has 79,308 complaints while Credit Card has only 35,645. This means Debt Collection is about 2.2 times more frequent. This is a problem because the model will be biased toward the majority class. We fix this later by downsampling.

5.4 Duplicate Narratives

Found 20,679 duplicate complaint narratives (about 18% of data). These are exact duplicates that need to be removed to avoid data leakage.

5.5 Redacted Text

Some complaints contain redacted information shown as "XXXX". We found 146 narratives where more than 50% of the text is redacted. These are not useful for classification and will be removed.

5.6 Word Count Analysis

Analyzed the length of complaint narratives. Average word count is about 186 words. Minimum is 5 words and maximum is 5,111 words. Very short narratives (under 20 words) do not have enough information for classification.

6. Notebook 2: Data Preprocessing

The second notebook (02_preprocessing.ipynb) is where we clean and prepare the data for machine learning. This is a critical step because garbage in means garbage out.

6.1 Step 1: Drop Unused Columns

We only keep the columns we need for classification: Product (target), Sub-product, Issue, Consumer complaint narrative (features), State, and Company. All other columns are dropped.

6.2 Step 2: Fill Null Values

For categorical columns (Sub-product, Issue, State, Company) that have null values, we fill them with the most common value (mode). For example, State had 284 nulls which were filled with "TX" (Texas) since it is the most common state.

6.3 Step 3: Drop Null Narratives

Any row without a complaint narrative is dropped. Since the narrative text is our main feature for classification, rows without it are useless.

6.4 Step 4: Remove Duplicates

Removed 20,679 duplicate narratives. This is important because having the same complaint in both training and test sets would give false high accuracy scores.

6.5 Step 5: Remove Heavily Redacted Narratives

Removed 144 narratives where more than 50% of the text is "XXXX" placeholders. These do not contain useful information for classification.

6.6 Step 6: Clean Text

Cleaned the complaint text by: removing markdown symbols (# and []), removing repeated asterisks (**), collapsing multiple spaces into one, and trimming whitespace.

6.7 Step 7: Filter Short Narratives

Removed 2,865 narratives that are shorter than 20 words. Very short complaints do not have enough context to be reliably classified.

6.8 Step 8: Balance Classes via Downsampling

This is the most important step. We had 57,756 Debt Collection and 33,509 Credit Card complaints. To balance them, we randomly selected 4,000 from each class, giving us a balanced dataset of 8,000 total complaints. This ensures the model learns equally from both categories.

6.9 Step 9: Train/Test Split

Split the balanced 8,000 complaints into 80% training (6,400) and 20% test (1,600). The training set is used to teach the model, and the test set is used to evaluate how well it performs on unseen data. We used stratified split to maintain equal class proportions in both sets.

7. Notebook 3: Model Training & Evaluation

The third notebook (03_model_training.ipynb) is where we build, train, and evaluate the machine learning models.

7.1 TF-IDF Vectorization

TF-IDF stands for Term Frequency-Inverse Document Frequency. It converts text into numbers that machine learning models can understand.

How it works: It looks at each word in a complaint and calculates a score based on how often that word appears in this complaint (term frequency) versus how often it appears across all complaints (inverse document frequency). Words that are common across all complaints get lower scores, while words that are specific to certain complaints get higher scores.

We set `max_features=5000`, meaning we only keep the top 5,000 most important words. We also used English stop words removal to ignore common words like "the", "is", "and".

7.2 Logistic Regression Model

We used Logistic Regression, a simple but effective classification algorithm. It works by learning a linear relationship between the TF-IDF features and the target category. Despite being simple, it performs very well on text classification.

The model was trained with `max_iter=1000` to ensure it converges, and `random_state=42` for reproducibility.

7.3 Google Gemini LLM Classification

We also tested Google Gemini 2.5 Flash, a Large Language Model, on a sample of 300 complaints. The LLM uses zero-shot classification - we simply give it the complaint text and ask it to classify it as "Debt Collection" or "Credit Card" without any training examples.

The LLM also provides reasoning for its classification, which makes it more explainable. However, it is slower and more expensive than the ML model.

8. Results and Performance Metrics

After training and evaluating both models, here are the results:

8.1 Traditional ML Model (Logistic Regression)

Metric	Score
Accuracy	93.67%
Precision	93.71%
Recall	94.30%
F1-Score	94.01%

8.2 Google Gemini LLM

The LLM metrics are currently set to 0 because the full evaluation has not been run with a valid API key yet. When run, it evaluates on a 300-row sample.

8.3 What These Metrics Mean

- Accuracy: 93.67% of all predictions were correct
- Precision: When the model predicts Debt Collection, it is correct 93.71% of the time
- Recall: The model correctly identifies 94.30% of all actual Debt Collection complaints
- F1-Score: The harmonic mean of Precision and Recall is 94.01%

8.4 Confusion Matrix

The confusion matrix shows the model made very few errors. Most complaints were correctly classified. The few errors were roughly evenly split between misclassifying Debt Collection as Credit Card and vice versa.

9. Web Application (Streamlit)

The Streamlit web application (app/app.py) provides a beautiful, dark-themed interface for users to interact with the classification system.

9.1 Features of the App

- Hero section with project description
- Text area to paste complaint narratives
- Side-by-side predictions from ML model and Gemini LLM
- Confidence scores for both predictions
- LLM provides reasoning for its classification
- Advanced Analytics section with:
 - Performance metrics comparison
 - Confusion matrix heatmap
 - Training visualizations (class balance, word counts)

9.2 How the App Works

1. User pastes a complaint narrative into the text area
2. User clicks "Analyze Complaint" button
3. The app loads the pre-trained ML model and TF-IDF vectorizer
4. The ML model transforms the text and makes a prediction
5. The app sends the text to Google Gemini API for LLM prediction
6. Both predictions are displayed side by side with confidence scores
7. User can expand Advanced Analytics to see detailed metrics

10. Technology Stack

Technology	Purpose
Python 3.13+	Core programming language
Pandas	Data manipulation and analysis
NumPy	Numerical computations
Scikit-learn	Machine learning (TF-IDF, Logistic Regression)
Google GenAI SDK	Google Gemini LLM integration
Streamlit	Web application framework
Matplotlib	Data visualization
Seaborn	Advanced statistical visualizations
Joblib	Model serialization (saving/loading)
Python-dotenv	Environment variable management

11. Key Interview Talking Points

11.1 What Problem Did You Solve?

Automated classification of consumer financial complaints into specific categories. The CFPB receives thousands of complaints daily, and manual classification is slow, expensive, and error-prone. Our system provides instant, accurate classification.

11.2 What Challenges Did You Face?

- Class imbalance: Debt Collection complaints were 2.2x more frequent than Credit Card
- Duplicate data: 18% of narratives were duplicates that needed removal
- Redacted text: Many complaints contained XXXX placeholders that are not useful
- Short narratives: Very short complaints lack context for reliable classification

11.3 How Did You Handle Class Imbalance?

Used stratified downsampling - randomly selected 4,000 samples from each class to create a perfectly balanced dataset of 8,000 total complaints. This ensures the model learns equally from both categories and does not become biased toward the majority class.

11.4 Why TF-IDF + Logistic Regression?

TF-IDF is a well-established text representation technique that captures the importance of words. Logistic Regression is fast, interpretable, and works very well for text classification. Together they achieve 93.67% accuracy while being much faster and cheaper than LLMs.

11.5 Why Compare ML vs LLM?

To demonstrate the trade-offs between traditional ML and modern LLMs. The ML model is fast (milliseconds), cheap (no API costs), and achieves high accuracy. The LLM is slower, more expensive, but provides reasoning and can handle edge cases better. In production, you might use the ML model for bulk processing and the LLM for ambiguous cases.

11.6 What Would You Do Differently?

- Try other models like Random Forest, XGBoost, or fine-tuned BERT
- Add more product categories beyond just Debt Collection and Credit Card
- Implement text preprocessing like lemmatization and n-grams
- Add a confidence threshold to route uncertain cases to human review

11.7 Key Numbers to Remember

- Raw data: 114,953 complaints
- After cleaning: 91,265 complaints
- Balanced sample: 8,000 (4,000 per class)
- Training set: 6,400 | Test set: 1,600
- Final accuracy: 93.67%
- Features: 5,000 TF-IDF features